

Day 6 Assignment

Designing a REST API with OpenAPI/Swagger Documentation

John Michael Bilbao
Techstart
August 13, 2026

Introduction

This assignment asks for a REST API for a book management system, documented with OpenAPI, with screenshots of each endpoint being exercised.

I wrote the OpenAPI document first and then implemented it, rather than the other way round. That order is the reason the two agree: the validation rules in the code are the limits written in the schemas, and the error shape the endpoints return is the one the document advertises. There is also a set of tests that reads openapi.yaml and checks the real responses against the schemas in it, so the two cannot quietly drift apart later.

The API is Flask, the document is OpenAPI 3.0.3, and the application serves both the specification at /openapi.yaml and Swagger UI at /docs. Every screenshot in this report is of that running API.

Step 1: The endpoints

Five endpoints, covering the four CRUD operations:

Method and path	What it does	Success	Failures
POST /books	Create a book	201	400, 409
GET /books	List every book, optionally filtered by author	200	—
GET /books/{id}	Retrieve one book	200	404
PUT /books/{id}	Update the fields given	200	400, 404, 409
DELETE /books/{id}	Delete a book	204	404

A few of those choices are worth explaining. POST returns 201 with a Location header pointing at the new book, so a client does not have to guess the URL it just created. DELETE returns 204 and no body, because there is nothing meaningful to send back. PUT changes only the fields present in the body, which means marking a book as out on loan is {"available": false} rather than a full copy of the record.

409 is there because a duplicate is not a malformed request. The body is perfectly valid; it conflicts with something already stored. Returning 400 for it would tell the client to fix its JSON, which is not the problem.

Every failure uses the same two-field shape, an error code and a sentence, so a client only has to learn one error format:

```
{
  "error": "not_found",
  "message": "No book with id 99."
}
```

Step 2: The OpenAPI document

openapi.yaml is about 260 lines and describes all five operations, the query parameter, the request bodies, every response, and four schemas: Book, NewBook, BookUpdate and Error. The POST operation as it appears in the file:

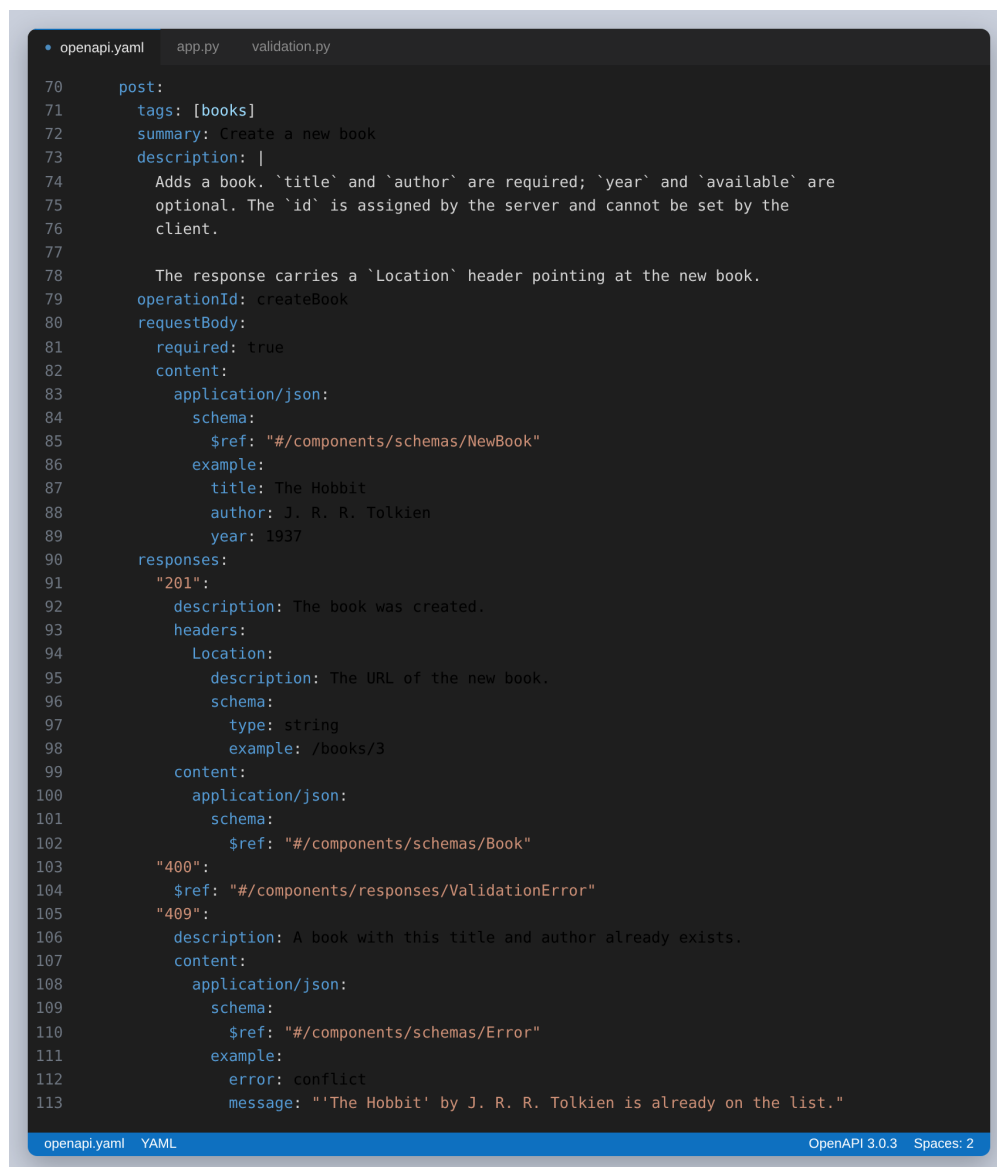


Figure 1: The POST /books operation in openapi.yaml

Three schemas rather than one is a deliberate choice. Book is what comes back and includes the server-assigned id. NewBook is what a client may send to create one, and has no id at all, which is how the document says that ids are not the client's to choose. BookUpdate is every writable field made optional with minProperties: 1, which is how it says that a partial update is fine but an empty body is not.

The repeated responses are written once under components and referenced. NotFound is defined in one place and pointed at from the three endpoints that can return it, so the description cannot end up saying three slightly different things.

Swagger UI reads that file directly. All five operations and the four schemas, as served at /docs:

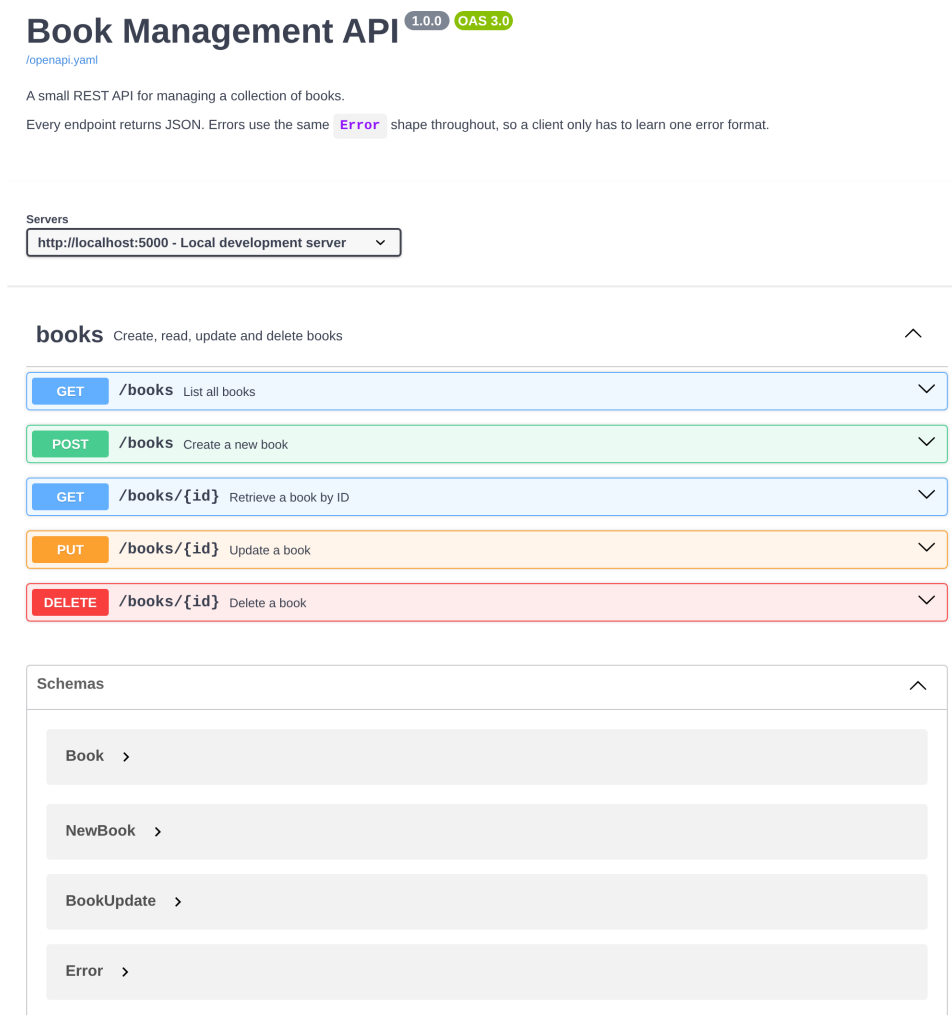


Figure 2: Swagger UI rendering openapi.yaml at /docs

Expanding POST /books shows what the document gives a reader: the required fields, the limits on each one, an example body, and all three responses it can produce with an example of each.

POST

/books Create a new book

Adds a book. **title** and **author** are required; **year** and **available** are optional. The **id** is assigned by the server and cannot be set by the client.

The response carries a **Location** header pointing at the new book.

Parameters

Cancel

No parameters

Request body required

application/json

```
{
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "year": 1937
}
```

Execute

Responses

Code	Description	Links						
201	The book was created. Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 1, "title": "Nineteen Eighty-Four", "author": "George Orwell", "year": 1949, "available": true }</pre> Headers: <table> <thead> <tr> <th>Name</th> <th>Description</th> <th>Type</th> </tr> </thead> <tbody> <tr> <td>Location</td> <td>The URL of the new book.</td> <td>string Example: /books/3</td> </tr> </tbody> </table>	Name	Description	Type	Location	The URL of the new book.	string Example: /books/3	No links
Name	Description	Type						
Location	The URL of the new book.	string Example: /books/3						
400	The body was missing something or had the wrong type. Media type application/json Example Value Schema <pre>{ "error": "validation_error", "message": "'title' is required." }</pre>	No links						
409	A book with this title and author already exists. Media type application/json Example Value Schema <pre>{ "error": "conflict", "message": "'The Hobbit' by J. R. R. Tolkien is already on the list." }</pre>	No links						

Figure 3: POST /books expanded, showing the request body and every documented response

The schemas at the bottom of the page are the same definitions the endpoints reference:

Schemas

Book ▾ {
 id*
 title*
 author*
 year
 available*
}

NewBook >

BookUpdate >

Error >

Figure 4: The Book schema, with the constraints the API enforces

Step 3: Testing each endpoint

Each endpoint below was called with curl against the running server. The -i flag prints the status line and headers as well as the body. The only thing changed in these screenshots is that the JSON body is indented; Flask sends it on one line, which runs off the edge of the page.

Creating a book. Note the 201 and the Location header:

```
books-api - POST /books

$ curl -i -X POST http://localhost:5000/books \
  -H 'Content-Type: application/json' \
  -d '{"title": "The Hobbit", "author": "J. R. R. Tolkien", "year": 1937}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 87
Location: /books/3
Connection: close

{
  "id": 3,
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "year": 1937,
  "available": true
}
```

Figure 5: POST /books

Listing everything. The two seeded books plus the one just created:

```
books-api - GET /books

$ curl -i http://localhost:5000/books
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 292
Connection: close

{
  "books": [
    {
      "id": 1,
      "title": "Nineteen Eighty-Four",
      "author": "George Orwell",
      "year": 1949,
      "available": true
    },
    {
      "id": 2,
      "title": "Brave New World",
      "author": "Aldous Huxley",
      "year": 1932,
      "available": true
    },
    {
      "id": 3,
      "title": "The Hobbit",
      "author": "J. R. R. Tolkien",
      "year": 1937,
      "available": true
    }
  ],
  "count": 3
}
```

Figure 6: GET /books

The documented query parameter, matching on part of the author's name, case-insensitively:

```
books-api - GET /books?author=orwell

$ curl -i 'http://localhost:5000/books?author=orwell'
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 116
Connection: close

{
  "books": [
    {
      "id": 1,
      "title": "Nineteen Eighty-Four",
      "author": "George Orwell",
      "year": 1949,
      "available": true
    }
  ],
  "count": 1
}
```

Figure 7: GET /books?author=orwell

Retrieving the new book by its id:

```
books-api - GET /books/{id}

$ curl -i http://localhost:5000/books/3
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 87
Connection: close

{
  "id": 3,
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "year": 1937,
  "available": true
}
```

Figure 8: GET /books/3

Updating one field. Only available was sent, and the title, author and year are unchanged in the response:

```
books-api - PUT /books/{id}

$ curl -i -X PUT http://localhost:5000/books/3 \
  -H 'Content-Type: application/json' \
  -d '{"available": false}'
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 88
Connection: close

{
  "id": 3,
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "year": 1937,
  "available": false
}
```

Figure 9: PUT /books/3

Deleting it. The 204 carries no body, and no Content-Type either, since there is no content to describe. Reading the same id afterwards gives the 404:

```
books-api - DELETE /books/{id}

$ curl -i -X DELETE http://localhost:5000/books/3
HTTP/1.1 204 NO CONTENT
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Connection: close

$ curl -i http://localhost:5000/books/3
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 53
Connection: close

{
  "error": "not_found",
  "message": "No book with id 3."
}
```

Figure 10: DELETE /books/3, then GET /books/3

Swagger UI can also call the API itself. This is Try it out on GET /books/{id}, executed against the running server, showing the request it sent and the response and headers that came back:

GET

/books/{id}

Retrieve a book by ID



Parameters

Cancel

Name	Description
id ★ required	The book's ID.
integer (path)	1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5000/books/1' \
-H 'accept: application/json'
```



Request URL

```
http://localhost:5000/books/1
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "title": "Nineteen Eighty-Four", "author": "George Orwell", "year": 1949, "available": true }</pre> Download Response headers <pre>connection: close content-length: 84 content-type: application/json date: Thu, 13 Aug 2026 06:07:38 GMT server: Werkzeug/3.1.8 Python/3.9.25</pre>

Figure 11: Try it out on GET /books/{id}, run against the live API

The error responses

The three failure cases, all returning the shape the document describes: a body with no title, a book that is already on the list, and an id that does not exist.

```
books-api - The three error responses

$ curl -i -X POST http://localhost:5000/books \
  -H 'Content-Type: application/json' \
  -d '{"author": "Nobody"}'
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 62
Connection: close

{
  "error": "validation_error",
  "message": "'title' is required."
}

$ curl -i -X POST http://localhost:5000/books \
  -H 'Content-Type: application/json' \
  -d '{"title": "Brave New World", "author": "Aldous Huxley"}'
HTTP/1.1 409 CONFLICT
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 92
Connection: close

{
  "error": "conflict",
  "message": "'Brave New World' by Aldous Huxley is already on the list."
}

$ curl -i http://localhost:5000/books/99
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Thu, 13 Aug 2026 06:07:32 GMT
Content-Type: application/json
Content-Length: 54
Connection: close

{
  "error": "not_found",
  "message": "No book with id 99."
}
```

Figure 12: 400, 409 and 404, all using the Error schema

The 400 names the field that was wrong rather than saying the request was invalid, which is the difference between a client developer fixing it in a minute and reading the spec line by line to work out which field the server disliked.

Keeping the document and the code in agreement

A specification that describes what the API was intended to do is worth very little. The part of this assignment I spent most time on was making the document checkable.

There are 48 tests. 34 cover behaviour. The other 14 load openapi.yaml and compare it against the running application:

```
books-api - test run

$ python3 -m pytest tests/ -q
..... [100%]
48 passed in 0.18s

$ python3 -m pytest tests/test_spec.py -v
test_spec_declares_the_five_endpoints PASSED [ 7%]
test_every_operation_has_a_summary_and_an_operation_id PASSED [ 14%]
test_every_documented_response_has_a_description PASSED [ 21%]
test_the_routes_the_app_serves_are_the_routes_the_spec_declares PASSED [ 28%]
test_list_response_matches_the_schema PASSED [ 35%]
test_create_response_matches_the_schema PASSED [ 42%]
test_a_book_with_no_year_still_matches_the_schema PASSED [ 50%]
test_get_one_response_matches_the_schema PASSED [ 57%]
test_update_response_matches_the_schema PASSED [ 64%]
test_delete_is_documented_as_having_no_body PASSED [ 71%]
test_every_error_response_matches_the_error_schema PASSED [ 78%]
test_error_codes_are_from_the_documented_enum PASSED [ 85%]
test_the_spec_is_served PASSED [ 92%]
test_swagger_ui_is_served_and_points_at_the_spec PASSED [100%]
===== 14 passed in 0.10s =====
```

Figure 13: The test run

Those 14 do four kinds of check. They confirm the document declares exactly the five operations and no more, and that each has a summary and an operationId. They walk Flask's own routing table and assert that every path the application serves under /books is one the document declares, so a route added in code and forgotten in the spec fails the build. They validate real responses against the response schemas with jsonschema, including a book with no year, since year is documented as nullable. And they check that every error response matches the Error schema and that the code is one of the three values in its enum.

Getting the schema validation working took one real piece of work. OpenAPI's nullable is not a JSON Schema keyword, so jsonschema ignores it, and a book with year: null failed against type: integer even though the document says null is allowed. The test helper resolves the \$refs and rewrites a nullable field's type into [type, "null"] before validating. Without that the tests would have quietly disagreed with the document they were meant to be enforcing.

What I would do differently

Writing the document first was the right call, and not because of the document. Deciding on paper that POST returns 201 with a Location header, that DELETE returns 204 with no body, and that a duplicate is a 409 rather than a 400, meant those questions were settled before any endpoint was written. When I have started from the code instead, the status codes end up being whatever the first implementation happened to return.

Two things I would change. The store is in memory, so the list resets when the server restarts; that was a deliberate trade to keep the endpoint code short, but it is the first thing I would replace. And PUT here updates only the fields it is given, which is really PATCH behaviour. The assignment specifies PUT, so PUT is what I documented and implemented, and the description says plainly that it replaces only the fields given, but on a real API I would either make PUT a full replacement

or move this behaviour to PATCH.